

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



ĐỀ TÀI 4

MÔN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Thuật toán Tối ưu Đàn Kiến (Ant Colony Optimization - ACO)

Họ và tên SV:

Hoàng Hữu Hiếu - 0207768 (Nhóm Trưởng)

Nguyễn Văn Minh - 0211068

Dương Trí Vũ - 0215568

Phạm Anh Toàn - 0295168

Mai Văn Thành - 0213068

Lớp học phần: 68CS1

Hà Nội, tháng 9/2025

Mục lục

1	Giới thiệu	2
2	Nguyên lý hoạt động	2
2.1	Ý tưởng sinh học	2
2.2	Tóm tắt công thức Ant Colony Optimization (ACO)	3
3	Các biến thể ACO	4
3.1	Ant System (AS)	4
3.2	Ant Colony System (ACS)	4
3.3	Max-Min Ant System (MMAS)	4
4	Ý tưởng chính	4
5	Các phần chính trong code	5
5.1	Khởi tạo dữ liệu	5
5.2	Cấu trúc một con kiến	5
5.3	Cập nhật pheromone	5
5.4	Tìm tour tốt nhất	5
5.5	Mã nguồn Python: ACO với Tkinter	5
5.6	Giao diện người dùng	16
6	So sánh với thuật toán khác	17
7	Ưu nhược điểm	17
7.1	Ưu điểm	17
7.2	Nhược điểm	18
8	Kết luận và hướng phát triển	19

1 Giới thiệu

Swarm Intelligence (SI) là lĩnh vực nghiên cứu các thuật toán phân tán lấy cảm hứng từ hành vi tập thể của các sinh vật tự nhiên. Một trong những đại diện nổi bật của SI là Ant Colony Optimization (ACO), do Dorigo và cộng sự đề xuất năm 1991 và phát triển trong thập niên 1990.

ACO ban đầu được áp dụng cho bài toán kinh điển Người bán hàng (Travelling Salesman Problem – TSP), nhưng sau đó đã được mở rộng ra nhiều lĩnh vực khác như lập lịch, routing mạng, logistics, và các bài toán NP-hard khác.

2 Nguyên lý hoạt động

2.1 Ý tưởng sinh học

Trong thế giới tự nhiên, những con kiến đầu tiên tìm đường lựa chọn ngẫu nhiên đường đi, và tìm được thức ăn trở lại tổ trong khi tìm đường chúng sẽ để lại pheromone của chúng trên đường đi để đánh dấu. Nếu những con kiến khác tìm kiếm một đường đi giống như vậy, chúng không phải di chuyển một cách ngẫu nhiên, mà thay vào đó là theo đường đi đã có trước đó, quay trở lại và củng cố lại nếu chúng tìm thấy được thức ăn. Tuy nhiên, theo thời gian pheromone được lưu lại trên đường đi sẽ bắt đầu bay hơi, do vậy làm giảm đi nồng độ của nó. Thời gian càng dài pheromone mà chúng để lại trên đoạn đường trước sẽ càng giảm lâu dần pheromone cũng sẽ bốc hơi hoàn toàn. Bằng cách so sánh, những con đường ngắn hơn sẽ lưu lại nồng độ pheromone cao hơn so với các con đường dài do có nhiều kiến đi qua hơn. Nếu pheromone bị bốc hơi hoàn toàn những con đường được chọn bởi những con kiến đầu tiên sẽ không còn thu hút đối với các con kiến sau đó. Trong trường hợp này ảnh hưởng của sự bốc hơi của pheromone có thể không thực sự ảnh hưởng tới việc tìm đường của đàn kiến nhưng nó có ý nghĩa rất quan trọng với các hệ thống nhân tạo.

Kết quả là khi các con kiến tìm thấy con đường tốt nhất tới nguồn thức ăn từ tổ của chúng thì những con kiến khác cũng sẽ di chuyển theo con đường đó và có những phản hồi tích cực về con đường dẫn đến việc các con kiến sau cũng sẽ đi theo một con đường duy nhất. Ý tưởng về giải thuật đàn kiến dựa trên hành vi với những kiến mà chúng ta giả định rằng chúng sẽ đi vòng quanh đồ thị để thấy được vấn đề cần giải quyết.

2.2 Tóm tắt công thức Ant Colony Optimization (ACO)

2.2.1. Xác suất chọn đường đi

Khi một con kiến đang ở đỉnh i , xác suất để nó chọn thành phố j tiếp theo được tính bởi:

$$P_{ij}(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{k \in \text{allowed}} [\tau_{ik}(t)]^\alpha \cdot [\eta_{ik}]^\beta}$$

Trong đó:

- $\tau_{ij}(t)$: lượng pheromone trên cạnh (i, j) tại thời điểm t .
- $\eta_{ij} = \frac{1}{d_{ij}}$: thông tin heuristic (ngược với khoảng cách).
- α : tham số điều chỉnh mức ảnh hưởng của pheromone.
- β : tham số điều chỉnh mức ảnh hưởng của heuristic.

2.2.2. Cập nhật pheromone

Sau mỗi vòng lặp, pheromone trên cạnh (i, j) được cập nhật theo công thức:

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij}(t)$$

Trong đó:

- $\rho \in (0, 1)$: hệ số bay hơi pheromone.
- $\Delta\tau_{ij}(t)$: lượng pheromone được các kiến bổ sung sau vòng lặp.

2.2.3. Lượng pheromone bổ sung

$$\Delta\tau_{ij}(t) = \sum_{k=1}^m \Delta\tau_{ij}^k(t)$$

Với từng con kiến k :

$$\Delta\tau_{ij}^k(t) = \begin{cases} \frac{Q}{L_k}, & \text{nếu kiến } k \text{ đi qua cạnh } (i, j) \\ 0, & \text{ngược lại} \end{cases}$$

Trong đó:

- Q : hằng số điều chỉnh lượng pheromone.
- L_k : độ dài hành trình mà kiến k tìm được.

2.2.4. Cơ chế ưu tú (Elitist Reinforcement)

Trong biến thể *Elitist Ant System*, ta cộng thêm pheromone trên hành trình tốt nhất từng tìm được:

$$\tau_{ij}(t+1) \leftarrow \tau_{ij}(t+1) + e \cdot \frac{Q}{L^*}$$

Trong đó:

- e : trọng số ưu tú (elitist weight).
- L^* : độ dài hành trình tốt nhất đã biết cho tới thời điểm t .

2.2.5. Ý nghĩa tham số

- α : ảnh hưởng của pheromone (ký ức tập thể).
- β : ảnh hưởng của heuristic (độ ngắn quãng đường).
- ρ : hệ số bay hơi pheromone.
- Q : lượng pheromone do mỗi kiến để lại.
- e : trọng số ưu tú (tăng cường đường đi tốt nhất).

3 Các biến thể ACO

3.1 Ant System (AS)

Phiên bản gốc, tất cả kiến đóng góp pheromone sau mỗi vòng lặp.

3.2 Ant Colony System (ACS)

Thêm cơ chế *cập nhật cục bộ* trong quá trình xây dựng tour để khuyến khích khám phá.

3.3 Max–Min Ant System (MMAS)

Giới hạn giá trị pheromone trong khoảng $[\tau_{\min}, \tau_{\max}]$ để tránh hội tụ sớm.

4 Ý tưởng chính

- **Bài toán:** Giải TSP (Travelling Salesman Problem) bằng Ant Colony Optimization (ACO).
- **Giao diện:** Tkinter + Matplotlib để vẽ đồ thị (tour tốt nhất, bản đồ pheromone, đường cong hội tụ).

- Người dùng có thể chỉnh tham số, chạy từng vòng hoặc nhiều vòng, reset dữ liệu.

5 Các phần chính trong code

5.1 Khởi tạo dữ liệu

- Sinh ngẫu nhiên n thành phố với tọa độ (x, y) .
- Tính ma trận khoảng cách $D[i, j]$.
- Khởi tạo pheromone ban đầu $pher[i, j] = 1$.

5.2 Cấu trúc một con kiến

- Xuất phát từ một thành phố ngẫu nhiên.
- Mỗi bước, xác suất chọn thành phố kế tiếp:

$$P(i \rightarrow j) \propto (\tau_{ij})^\alpha \cdot \left(\frac{1}{d_{ij}}\right)^\beta$$

- Lặp lại cho đến khi thăm hết tất cả các thành phố $\rightarrow$ tạo tour.

5.3 Cập nhật pheromone

- Bay hơi: $pher* = (1 - \rho)$.
- Mỗi kiến đi qua cạnh $(i, j) \rightarrow$ cộng thêm Q/L_k .
- Đường đi tốt nhất toàn cục có thêm “elitist reinforcement”.

5.4 Tìm tour tốt nhất

- Sau mỗi vòng, so sánh độ dài tour các kiến.
- Lưu trữ tour tốt nhất toàn cục ($best_path, best_len$).

5.5 Mã nguồn Python: ACO với Tkinter

Dưới đây là toàn bộ code:

```
1 import tkinter as tk
2 from tkinter import ttk
3 import numpy as np
4 import matplotlib
```

```

5 matplotlib.use("TkAgg")
6 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
7 import matplotlib.pyplot as plt
8
9 def build_distance_matrix(pts):
10     n = len(pts)
11     D = np.zeros((n, n), dtype=float)
12     for i in range(n):
13         for j in range(i+1, n):
14             d = np.linalg.norm(pts[i]-pts[j])
15             D[i, j] = d
16             D[j, i] = d
17     np.fill_diagonal(D, 1e-9)
18     return D
19
20 def path_length(path, D):
21     total = 0.0
22     for i in range(len(path)):
23         a = path[i]; b = path[(i+1) % len(path)]
24         total += D[a, b]
25     return total
26
27 def construct_ant_full(D, pher, alpha, beta, rng):
28     n = D.shape[0]
29     start = rng.integers(0, n)
30     unvisited = list(range(n)); unvisited.remove(start)
31     tour = [start]
32     while unvisited:
33         i = tour[-1]
34         desir = []
35         for j in unvisited:
36             tau = pher[i, j] ** alpha
37             eta = (1.0 / D[i, j]) ** beta
38             desir.append(tau * eta)
39         p = np.array(desir) / np.sum(desir)
40         r = rng.random()
41         cum = np.cumsum(p)
42         idx = int(np.searchsorted(cum, r))
43         tour.append(unvisited.pop(idx))
44     return tour
45

```

```

46 def choose_next_city(i, unvisited, D, pher, alpha, beta, rng):
47     desir = []
48     for j in unvisited:
49         tau = pher[i, j] ** alpha
50         eta = (1.0 / D[i, j]) ** beta
51         desir.append(tau * eta)
52     p = np.array(desir) / np.sum(desir)
53     r = rng.random()
54     cum = np.cumsum(p)
55     idx = int(np.searchsorted(cum, r))
56     return unvisited.pop(idx)
57
58 class ACOAppVNAnim:
59     def __init__(self, root):
60         self.root = root
61         root.title("ACO cho TSP      Tkinter (VI) | Animation demo"
62             )
63         self.rng = np.random.default_rng(123)
64
65         self.defaults = {
66             "n": 30, "ants": 60, "alpha": 1.0, "beta": 4.0,
67             "rho": 0.5, "Q": 100.0, "elitist": 2.0, "steps": 10
68         }
69
70         ctrl = ttk.Frame(root); ctrl.pack(side=tk.LEFT, fill=tk.Y,
71             padx=8, pady=8)
72
73         self.n_var = tk.IntVar(value=self.defaults["n"])
74         self.ants_var = tk.IntVar(value=self.defaults["ants"])
75         self.alpha_var = tk.DoubleVar(value=self.defaults["alpha"
76             ])
77         self.beta_var = tk.DoubleVar(value=self.defaults["beta"])
78         self.rho_var = tk.DoubleVar(value=self.defaults["rho"])
79         self.Q_var = tk.DoubleVar(value=self.defaults["Q"])
80         self.elite_var = tk.DoubleVar(value=self.defaults["elitist
81             "])
82         self.steps_var = tk.IntVar(value=self.defaults["steps"])
83
84         ttk.Label(ctrl, text="S      th nh ph      (n g u nhi n)").
85             grid(row=0, column=0, sticky="w")
86         ttk.Spinbox(ctrl, from_=5, to=200, textvariable=self.n_var

```



```

, width=8).grid(row=0, column=1, sticky="w")
82 ttk.Button(ctrl, text=" T o d l i u n g u n h i n ",
    command=self.generate).grid(row=0, column=2, padx=6,
    pady=4, sticky="w")
83
84 row = 1
85 def add_param(label, var, vmin, vmax, increment=0.1,
    is_int=False):
86     nonlocal row
87     ttk.Label(ctrl, text=label).grid(row=row, column=0,
        sticky="w", pady=2)
88     scale = ttk.Scale(ctrl, from_=vmin, to=vmax, variable=
        var, orient=tk.HORIZONTAL)
89     scale.grid(row=row, column=1, sticky="we", padx=(6,6))
90     if is_int:
91         sp = ttk.Spinbox(ctrl, from_=vmin, to=vmax,
            textvariable=var, width=8)
92     else:
93         sp = ttk.Spinbox(ctrl, from_=vmin, to=vmax,
            increment=increment, textvariable=var, width=8)
94     sp.grid(row=row, column=2, sticky="w")
95     def on_var_change(*_):
96         try: v = var.get()
97         except tk.TclError: return
98         if is_int: sp.delete(0,"end"); sp.insert(0, f"{int
            (float(v))}")
99         else: sp.delete(0,"end"); sp.insert(0, f"{float(v)
            :.2f}")
100     var.trace_add("write", on_var_change); on_var_change()
101     row += 1
102
103 add_param(" S k i n m i v n g l p ", self.ants_var,
    1, 300, is_int=True)
104 add_param("Alpha ( n h h n g pheromone)", self.
    alpha_var, 0.0, 5.0, increment=0.1)
105 add_param("Beta ( n h h n g k h o n g c c h)", self.
    beta_var, 0.0, 10.0, increment=0.1)
106 add_param(" H s b a y h i (rho)", self.rho_var,
    0.0, 1.0, increment=0.01)
107 add_param(" H s t n g c n g Q", self.Q_var, 1.0,
    500.0, increment=1.0)

```

```

108     add_param(" T r n g   s   u   t   (elitist)", self.elite_var
109             , 0.0, 10.0, increment=0.1)
110
111     ttk.Label(ctrl, text=" C h y   N   v   n g   l   p ").grid(row=row,
112             column=0, sticky="w", pady=(6,2))
113
114     ttk.Spinbox(ctrl, from_=1, to=5000, textvariable=self.
115             steps_var, width=8).grid(row=row, column=1, sticky="w")
116     row += 1
117
118     # Animation controls
119     self.anim_on = tk.BooleanVar(value=True)
120     self.anim_delay = tk.IntVar(value=200) # ms per hop
121     animf = ttk.Frame(ctrl); animf.grid(row=row, column=0,
122             columnspan=3, sticky="we", pady=6)
123     ttk.Checkbutton(animf, text=" B t   a n i m a t i o n   ( k i n   b   ) "
124             , variable=self.anim_on).pack(side=tk.LEFT)
125     ttk.Label(animf, text=" T c           (ms/ b   c ):").pack(
126             side=tk.LEFT, padx=(8,2))
127     ttk.Spinbox(animf, from_=10, to=2000, increment=10,
128             textvariable=self.anim_delay, width=7).pack(side=tk.
129             LEFT)
130     row += 1
131
132     btns = ttk.Frame(ctrl); btns.grid(row=row, column=0,
133             columnspan=3, sticky="we", pady=6)
134     ttk.Button(btns, text=" M c           n h           t h a m   s   ", command=
135             self.set_defaults).pack(fill=tk.X, pady=2)
136     ttk.Button(btns, text="   t           l i   ", command=self.
137             reset_full).pack(fill=tk.X, pady=2)
138     ttk.Button(btns, text=" C h y   1   v   n g           ", command=
139             self.step).pack(fill=tk.X, pady=2)
140     ttk.Button(btns, text=" C h y   1   v   n g   ( a n i m a t e )           ",
141             command=self.step_animated).pack(fill=tk.X, pady=2)
142     ttk.Button(btns, text=" C h y   N   v   n g           ", command=self.
143             runN).pack(fill=tk.X, pady=2)
144     ttk.Button(btns, text=" D n g           ", command=self.
145             stop_animation).pack(fill=tk.X, pady=2)
146     row += 1
147
148     ctrl.columnconfigure(1, weight=1)
149

```

```

134     nb = ttk.Notebook(root); nb.pack(side=tk.RIGHT, fill=tk.
        BOTH, expand=True)
135     self.frame_tour = ttk.Frame(nb); nb.add(self.frame_tour,
        text="H nh tr nh t t n h t ")
136     self.frame_phero = ttk.Frame(nb); nb.add(self.frame_phero,
        text=" B n pheromone")
137     self.frame_conv = ttk.Frame(nb); nb.add(self.frame_conv,
        text=" H i t ")
138
139     self.fig1, self.ax1 = plt.subplots()
140     self.canvas1 = FigureCanvasTkAgg(self.fig1, master=self.
        frame_tour)
141     self.canvas1.get_tk_widget().pack(fill=tk.BOTH, expand=
        True)
142
143     self.fig2, self.ax2 = plt.subplots()
144     self.canvas2 = FigureCanvasTkAgg(self.fig2, master=self.
        frame_phero)
145     self.canvas2.get_tk_widget().pack(fill=tk.BOTH, expand=
        True)
146
147     self.fig3, self.ax3 = plt.subplots()
148     self.canvas3 = FigureCanvasTkAgg(self.fig3, master=self.
        frame_conv)
149     self.canvas3.get_tk_widget().pack(fill=tk.BOTH, expand=
        True)
150
151     self.points = None; self.D = None; self.pher = None
152     self.best_path = None; self.best_len = np.inf
153     self.history = []
154
155     self._anim_job = None
156     self._anim_state = None
157
158     self.generate()
159
160     def set_defaults(self):
161         self.ants_var.set(self.defaults["ants"])
162         self.alpha_var.set(self.defaults["alpha"])
163         self.beta_var.set(self.defaults["beta"])
164         self.rho_var.set(self.defaults["rho"])

```

```

165         self.Q_var.set(self.defaults["Q"])
166         self.elite_var.set(self.defaults["elitist"])
167         self.steps_var.set(self.defaults["steps"])
168
169     def reset_full(self):
170         if self.points is None:
171             self.generate(); return
172         n = len(self.points)
173         self.pher = np.ones((n, n), dtype=float)
174         np.fill_diagonal(self.pher, 1e-9)
175         self.best_path = None; self.best_len = np.inf
176         self.history = []
177         self.stop_animation()
178         self.draw_all()
179
180     def generate(self):
181         n = self.n_var.get()
182         self.points = self.rng.uniform(0, 100, size=(n,2)).astype(
183             float)
184         self.D = build_distance_matrix(self.points)
185         self.pher = np.ones((n, n), dtype=float)
186         np.fill_diagonal(self.pher, 1e-9)
187         self.best_path = None; self.best_len = np.inf
188         self.history = []
189         self.stop_animation()
190         self.draw_all()
191
192     def update_pheromones(self, ant_paths, lengths, rho, Q,
193         elitist):
194         self.pher *= (1.0 - rho)
195         for tour, L in zip(ant_paths, lengths):
196             dep = Q / L
197             for i in range(len(tour)):
198                 a = tour[i]; b = tour[(i+1) % len(tour)]
199                 self.pher[a,b] += dep; self.pher[b,a] += dep
200             if self.best_path is not None and elitist > 0:
201                 Lb = self.path_length(self.best_path)
202                 dep = elitist * (Q / Lb)
203                 for i in range(len(self.best_path)):
204                     a = self.best_path[i]; b = self.best_path[(i+1) %
205                         len(self.best_path)]

```

```

203         self.pher[a,b] += dep; self.pher[b,a] += dep
204     self.pher = np.clip(self.pher, 1e-9, None)
205
206     def path_length(self, path):
207         return path_length(path, self.D)
208
209     def step(self):
210         if self.points is None: return
211         ants = int(self.ants_var.get())
212         alpha = float(self.alpha_var.get())
213         beta = float(self.beta_var.get())
214         rho = float(self.rho_var.get())
215         Q = float(self.Q_var.get())
216         elitist = float(self.elite_var.get())
217
218         ant_paths, lengths = [], []
219         for _ in range(ants):
220             tour = construct_ant_full(self.D, self.pher, alpha,
221                                     beta, self.rng)
222             L = self.path_length(tour)
223             ant_paths.append(tour); lengths.append(L)
224             lengths = np.array(lengths)
225
226             idx = int(np.argmin(lengths))
227             if lengths[idx] < self.best_len:
228                 self.best_len = float(lengths[idx])
229                 self.best_path = ant_paths[idx][:]
230
231         self.update_pheromones(ant_paths, lengths, rho, Q, elitist
232                               )
233
234         it = len(self.history) + 1
235         self.history.append((it, float(self.best_len)))
236
237         self.draw_all()
238
239     def step_animated(self):
240         if not self.anim_on.get():
241             self.step(); return
242         if self.points is None: return
243         if self._anim_job is not None:

```

```

242         return
243     n = self.D.shape[0]
244     start = self.rng.integers(0, n)
245     unvisited = list(range(n)); unvisited.remove(start)
246     self._anim_state = {"tour": [start], "unvisited": unvisited,
247                         "lines": []}
247     self.ax1.clear()
248     pts = self.points
249     self.ax1.scatter(pts[:,0], pts[:,1])
250     self.ax1.set_title(" ang  animate  k i n      i      d i n ... "
251                        )
251     self.ax1.set_xlabel("x"); self.ax1.set_ylabel("y")
252     self.canvas1.draw()
253     self._animate_step()
254
255     def _animate_step(self):
256         st = self._anim_state
257         if st is None: return
258         pts = self.points; D = self.D; pher = self.pher
259         alpha = float(self.alpha_var.get()); beta = float(self.
260                    beta_var.get())
261         if not st["unvisited"]:
262             a = st["tour"][-1]; b = st["tour"][0]
263             line, = self.ax1.plot([pts[a,0], pts[b,0]], [pts[a,1],
264                    pts[b,1]], linewidth=2)
265             st["lines"].append(line)
266             self.canvas1.draw()
267             self._finish_animation_iteration(st["tour"])
268             self._anim_state = None; self._anim_job = None
269             return
270         i = st["tour"][-1]
271         j = choose_next_city(i, st["unvisited"], D, pher, alpha,
272                    beta, self.rng)
273         st["tour"].append(j)
274         line, = self.ax1.plot([pts[i,0], pts[j,0]], [pts[i,1], pts
275                    [j,1]], linewidth=2)
276         st["lines"].append(line)
277         self.canvas1.draw()
278         delay = int(self.anim_delay.get())
279         self._anim_job = self.root.after(delay, self._animate_step
280                    )

```

```

276
277 def _finish_animation_iteration(self, rep_tour):
278     ants = int(self.ants_var.get())
279     alpha = float(self.alpha_var.get()); beta = float(self.
        beta_var.get())
280     rho = float(self.rho_var.get()); Q = float(self.Q_var.get
        ())
281     elitist = float(self.elite_var.get())
282     ant_paths = [rep_tour]; lengths = [self.path_length(
        rep_tour)]
283     for _ in range(ants-1):
284         tour = construct_ant_full(self.D, self.pher, alpha,
            beta, self.rng)
285         L = self.path_length(tour)
286         ant_paths.append(tour); lengths.append(L)
287     lengths = np.array(lengths)
288     idx = int(np.argmin(lengths))
289     if lengths[idx] < self.best_len:
290         self.best_len = float(lengths[idx]); self.best_path =
            ant_paths[idx][:]
291     self.update_pheromones(ant_paths, lengths, rho, Q, elitist
        )
292     it = len(self.history) + 1; self.history.append((it, float
        (self.best_len)))
293     self.draw_all()
294
295 def stop_animation(self):
296     if self._anim_job is not None:
297         try: self.root.after_cancel(self._anim_job)
298         except Exception: pass
299     self._anim_job = None; self._anim_state = None
300
301 def runN(self):
302     for _ in range(int(self.steps_var.get())):
303         self.step()
304
305 def draw_best_tour(self):
306     self.ax1.clear()
307     pts = self.points
308     self.ax1.scatter(pts[:,0], pts[:,1])
309     if self.best_path is not None:

```

```

310         t = self.best_path
311         for i in range(len(t)):
312             a = t[i]; b = t[(i+1)%len(t)]
313             self.ax1.plot([pts[a,0], pts[b,0]], [pts[a,1], pts
314                           [b,1]], linewidth=2)
315             self.ax1.set_title(f"          d i t t n h t : {self.
316                               best_len:.3f}")
317         else:
318             self.ax1.set_title("Ch a c h n h t r n h t t
319                               n h t ")
320         self.ax1.set_xlabel("x"); self.ax1.set_ylabel("y")
321         self.canvas1.draw()
322
323     def draw_pheromone_heat(self):
324         self.ax2.clear()
325         pts = self.points; n = len(pts)
326         P = self.pher.copy()
327         P = P / (P.max() if P.max() > 0 else 1.0)
328         for i in range(n):
329             for j in range(i+1, n):
330                 w = P[i, j]
331                 if w > 0.01:
332                     self.ax2.plot([pts[i,0], pts[j,0]], [pts[i,1],
333                           pts[j,1]], linewidth=1 + 3*w)
334         self.ax2.scatter(pts[:,0], pts[:,1])
335         self.ax2.set_title(" B n          pheromone ( c n h d y =
336                           m i m n h )")
337         self.ax2.set_xlabel("x"); self.ax2.set_ylabel("y")
338         self.canvas2.draw()
339
340     def draw_convergence(self):
341         self.ax3.clear()
342         if self.history:
343             hist = np.array(self.history, dtype=float)
344             self.ax3.plot(hist[:,0], hist[:,1], marker="o")
345             self.ax3.set_xlabel("v n g l p ")
346             self.ax3.set_ylabel("          d i t t n h t ")
347             self.ax3.set_title(" B i u          h i t (best-so-
348                               far)")
349         else:
350             self.ax3.set_title(" C h y v i v n g          xem

```



```

345         ng cong h i t ")
346         self.canvas3.draw()
347
348     def draw_all(self):
349         self.draw_best_tour()
350         self.draw_pheromone_heat()
351         self.draw_convergence()
352
353 if __name__ == "__main__":
354     root = tk.Tk()
355     app = ACOAppVNAAnim(root)
356     root.mainloop()

```

5.6 Giao diện người dùng

- Thanh tiêu đề:

- Hiển thị tên chương trình: *ACO cho TSP – Tkinter (VI) | Animation nhiều vòng.*
- Cho biết ứng dụng dùng thuật toán ACO cho bài toán TSP, có hỗ trợ hiển thị động nhiều vòng lặp.

- Khu vực điều khiển tham số (bên trái):

- *Số thành phố (ngẫu nhiên)*: chọn số lượng điểm (thành phố).
- *Tạo dữ liệu ngẫu nhiên*: sinh ngẫu nhiên các tọa độ thành phố.
- *Số kiến mỗi vòng lặp*: số lượng kiến trong một vòng lặp.
- *Alpha (ảnh hưởng pheromone)*: hệ số điều chỉnh tầm quan trọng pheromone.
- *Beta (ảnh hưởng khoảng cách)*: hệ số điều chỉnh tầm quan trọng của khoảng cách.
- *Hệ số bay hơi ρ* : mức độ bay hơi pheromone qua từng vòng.
- *Hệ số tăng cường Q* : lượng pheromone tăng thêm theo chất lượng hành trình.
- *Trọng số ưu tú (elitist)*: hệ số tăng cường pheromone cho hành trình tốt nhất.
- *Chạy N vòng lặp*: số vòng lặp tối đa của thuật toán.
- *Tốc độ (ms/bước)*: tốc độ hiển thị animation.

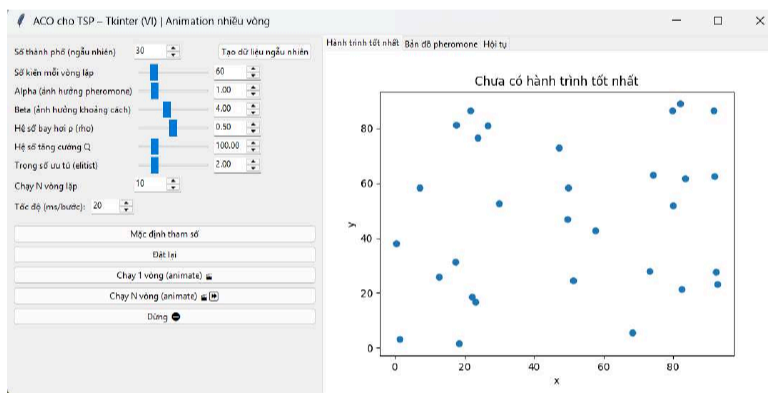
- Các nút chức năng:

- *Mặc định tham số*: đặt lại giá trị tham số về mặc định.

- *Đặt lại*: khởi động lại dữ liệu và tiến trình.
- *Chạy 1 vòng (animate)*: chạy một vòng lặp và hiển thị kết quả.
- *Chạy N vòng (animate)*: chạy liên tục nhiều vòng.
- *Dừng*: dừng animation đang chạy.

- **Khu vực hiển thị kết quả (bên phải):** gồm 3 tab chính

- *Hành trình tốt nhất*: biểu diễn hành trình tối ưu hiện tại; khi chưa có lời giải sẽ hiển thị “Chưa có hành trình tốt nhất”.
- *Bản đồ pheromone*: thể hiện mật độ pheromone trên các cạnh.
- *Hội tụ*: biểu đồ theo dõi quá trình hội tụ của độ dài hành trình theo số vòng.



Hình 1: Giao diện người dùng chương trình ACO cho TSP

6 So sánh với thuật toán khác

- So với PSO: PSO mạnh hơn với tối ưu liên tục, ACO tốt cho tối ưu rời rạc.
- So với GA: GA dựa trên tiến hóa, dễ kết hợp với ACO (hybrid ACO-GA).

7 Ưu nhược điểm

7.1 Ưu điểm

- **Tính hiệu quả với bài toán tổ hợp:** ACO đặc biệt phù hợp để giải các bài toán tối ưu rời rạc phức tạp như TSP, lập lịch, phân công, hay routing mạng.

- **Khả năng hội tụ đến nghiệm tốt:** Nhờ cơ chế pheromone tích lũy và heuristic, ACO có xu hướng hội tụ đến nghiệm gần tối ưu trong thời gian hợp lý.
- **Khả năng song song hóa cao:** Các con kiến hoạt động độc lập khi xây dựng hành trình, do đó có thể dễ dàng triển khai trên hệ thống đa luồng hoặc phân tán để tăng tốc tính toán.
- **Tính linh hoạt và mở rộng:** Có thể điều chỉnh, lai ghép hoặc kết hợp ACO với các thuật toán khác (GA, PSO, Tabu Search, Simulated Annealing, v.v.) để nâng cao hiệu quả.
- **Khả năng thích ứng động:** ACO có thể ứng dụng trong môi trường thay đổi (dynamic optimization), ví dụ như routing trong mạng máy tính, nơi mà cấu trúc mạng thay đổi theo thời gian.
- **Cơ chế tìm kiếm cân bằng:** Vừa khai thác (exploitation – dựa trên pheromone tích lũy) vừa thăm dò (exploration – nhờ yếu tố ngẫu nhiên), giúp tránh kẹt ở nghiệm cục bộ.

7.2 Nhược điểm

- **Nhạy cảm với tham số:** Hiệu quả của thuật toán phụ thuộc mạnh vào việc lựa chọn các tham số ($\alpha, \beta, \rho, Q, elitist$). Nếu chọn sai, thuật toán dễ hội tụ chậm hoặc bị kẹt ở nghiệm kém.
- **Tổn chi phí tính toán:** Với số lượng thành phố/phần tử lớn, số lượng kiến và vòng lặp tăng cao dẫn đến chi phí tính toán lớn, nhất là trong các ứng dụng thời gian thực.
- **Dễ rơi vào hội tụ sớm:** Nếu không có cơ chế kiểm soát (như giới hạn pheromone trong MMAS), pheromone có thể tập trung quá nhanh vào một số cạnh, làm thuật toán mất khả năng khám phá.
- **Khó phân tích lý thuyết:** Việc chứng minh tính hội tụ hoặc ước lượng độ phức tạp lý thuyết của ACO không dễ dàng, chủ yếu vẫn dựa vào thực nghiệm.
- **Khó mở rộng trực tiếp sang bài toán liên tục:** ACO ban đầu được thiết kế cho không gian rời rạc, để áp dụng vào tối ưu liên tục cần biến đổi hoặc kết hợp với các phương pháp khác.
- **Phụ thuộc ngẫu nhiên:** Do có yếu tố ngẫu nhiên, kết quả mỗi lần chạy có thể khác nhau; để đảm bảo chất lượng nghiệm cần chạy nhiều lần và lấy nghiệm tốt nhất.

8 Kết luận và hướng phát triển

ACO là một metaheuristic mạnh mẽ cho các bài toán NP-hard. Hướng phát triển tiếp theo bao gồm:

- Tích hợp ACO với học sâu.
- Áp dụng ACO vào routing mạng thể hệ mới.
- Kết hợp với các thuật toán khác để cải thiện tốc độ hội tụ.

Tài liệu

- [1] Dorigo M., Maniezzo V., Colorni A., “Ant System: Optimization by a colony of cooperating agents,” IEEE Transactions on Systems, Man, and Cybernetics, 1996.
- [2] Dorigo M., Di Caro G., “The Ant Colony Optimization Meta-Heuristic,” in New Ideas in Optimization, McGraw-Hill, 1999.
- [3] Datacamp, “Swarm Intelligence Algorithms,”
<https://www.datacamp.com/tutorial/swarm-intelligence>, 2024.
- [4] Viblo, “Tìm hiểu đôi chút về giải thuật đàn kiến,”
<https://viblo.asia/p/tim-hieu-doi-chut-ve-giai-thuat-dan-kien>, 2023.
- [5] Akavall, GitHub repo *AntColonyOptimization*,
<https://github.com/Akavall/AntColonyOptimization>.